

Machine Learning for Forecasting Hazardous Air Quality Days in Indian Cities

Jaykumar Vinodbhai Chauhan

Student ID: 34710280

Master of Mathematics

MTH5000 Masters Research Project

Supervisor: Dr Tianhai Tian

School of Mathematics, Monash University

October 2, 2026

Abstract

Air pollution in Indian cities regularly reaches hazardous concentrations, and a forecast is useful only if it arrives before the hazardous day does. This project forecasts daily PM_{2.5} one to three days ahead at a Delhi monitor and evaluates the forecasts as a warning system rather than by average error alone. The central finding is that the alarm budget a decision maker accepts dominates the choice of model. Across ten specifications, from simple benchmarks and a classical ARIMA model with annual Fourier terms to four machine learning regressors and a recurrent network, the spread in hit rate at a fixed alarm budget is about five percentage points, while moving the budget from forty to one hundred alarms a year moves the hit rate by fifty. Machine learning earns its place only partially. The classical model beats every machine learning regressor at every horizon, a result strengthened rather than weakened by a sensitivity analysis on the cleaning threshold, and the one model that does win, a recurrent network, wins only at one day ahead, reported as a range across seeds rather than a single favourable one. Two real defects in the source monitor, a clipped instrument ceiling and a miscalculated coverage statistic, were diagnosed and corrected before modelling began. The recommendation is to design a warning system around its alarm budget, treating the forecasting model as a secondary refinement.

Contents

Abstract	i
1 Introduction	1
1.1 Research Background	1
1.2 Research Problem	1
1.3 Research Objectives	2
1.4 Research Questions	2
1.5 Research Significance	2
1.6 Structure of the Report	2
2 Literature Review	4
2.1 Air Quality Forecasting: Challenges and Characteristics	4
2.2 Classical and Machine Learning Methods for Time Series Forecasting	4
2.3 Deep Learning Methods for Time Series Forecasting	5
2.3.1 Recurrent Neural Networks and the Vanishing Gradient Problem	5
2.3.2 The Gated Recurrent Unit	5
2.4 Machine Learning Applications to Delhi and Indian Air Quality	5
2.5 Forecast Verification as a Warning System	6
2.6 Data Quality in Reference Air Quality Monitors	6
2.7 Research Gap and Chapter Summary	6
3 Methodology	8
3.1 Research Design	8
3.2 Data Collection and Station Selection	8
3.3 Data Preprocessing	9
3.3.1 Data Cleaning: Two Defects in the Source Data	9
3.3.2 Seasonality Analysis	10

3.3.3	Feature Construction	12
3.4	Machine Learning Methods	12
3.4.1	Classical Benchmark: ARIMA with Fourier Terms	12
3.4.2	Penalised Linear Regression	13
3.4.3	Random Forest	13
3.4.4	Histogram Gradient Boosting	13
3.5	Deep Learning Methods	14
3.5.1	The Gated Recurrent Unit: Architecture and Training	14
3.6	Rolling-Origin Evaluation	15
3.7	Evaluation Measures	15
3.8	Reproducibility	16
3.9	Chapter Summary	16
4	Results and Analysis	17
4.1	Forecast Accuracy	17
4.2	Two Arms of the Same Question	18
4.3	The Decision Threshold	18
4.4	Chapter Summary	20
5	Discussion	21
5.1	The Operational Question Dominates the Modelling Question	21
5.2	How Large Is a Difference Between Models?	21
5.3	Limitations	22
6	Conclusion	24
6.1	Summary of the Study	24
6.2	Main Conclusions	24
6.3	Future Work	25

List of Figures

3.1	The coverage defect. Panel (a) shows the number of readings contributing to each daily mean, with the cadence estimated from the data each year as the solid line and the fixed denominator of 24 as the dashed line. Panel (b) shows the rate of implausible values by true coverage.	10
3.2	The cleaned daily series, with days exceeding the threshold shaded. The episodic structure is the feature the project sets out to forecast: exceedances arrive as multi-day winter episodes rather than as isolated days.	10
3.3	No weekly cycle. Lags 7, 14 and 21 are circled and lie on the decay curve rather than above it. Every day of week interval contains zero.	11
3.4	The annual cycle, by contrast, is unmistakable: monthly median with the interquartile range shaded. Its strength is what makes the absence in Figure 3.3 credible rather than an artefact of a weak test.	11
4.1	The trade-off, by horizon. The top row shows hit rate against false alarm rate; the bottom row replaces the false alarm rate with alarms raised per year, the unit in which an operator budgets. The curves very nearly coincide, which is the finding. They have deliberately not been separated for legibility.	19

List of Tables

3.1	The analysis dataset after cleaning.	9
3.2	Cleaning rules applied, and the basis for each.	9
4.1	Relative mean absolute error against persistence, under rolling origin evaluation over 2,303 origins. Lower is better and 1.000 is persistence. The recurrent network is given as the range over five initialisation seeds; it is the only model whose result depends on one.	17
4.2	Best hit rate available within a budget of alarms per year, with the event fixed at the health threshold and only the decision threshold varying.	18

Chapter 1

Introduction

1.1 Research Background

Air pollution in Indian cities regularly reaches concentrations classified as hazardous to human health. The public response to such episodes depends on knowing that a severe day is coming before it arrives: a forecast issued on the morning of a hazardous day is of limited value, while one issued a day or more ahead can change behaviour, whether through health advisories, restrictions on outdoor activity, or school closures. Daily pollution series carry structure that makes such forecasting feasible in principle: persistence from one day to the next, a strong annual cycle driven by meteorology and seasonal agricultural burning, and dependence on recent local conditions.

My interest in this question is also personal. Having grown up in India, air pollution was not an abstract statistic but part of ordinary life: seasonal smog that turned daylight hazy, and days when going outside meant a sore throat. Whether those days can be predicted reliably enough to be worth warning people about is a question I have a genuine stake in.

1.2 Research Problem

Existing machine learning studies of Indian air quality (reviewed in Chapter 2) report forecast accuracy by average error, and do not decompose that accuracy by the operating point at which a warning would actually be raised. A model with respectable average error can still be a poor warning system if it systematically misses the extreme days that are the only ones anyone cares about, and the reverse also holds: two models with very different average error can produce nearly identical warning performance once a decision threshold is applied. Separately, the project this report was proposed under assumed that daily pollution series carry a weekly cycle alongside the annual one, driven by traffic and industrial activity, and treated reconciling two seasonal periods as a central modelling difficulty. Neither assumption is safe to carry into the modelling without testing it directly, and both are tested in this report.

1.3 Research Objectives

1. To develop and evaluate a suite of classical, machine learning, and deep learning models for forecasting daily PM_{2.5} concentrations one to three days ahead at a single, well-characterised Indian monitoring station, under an evaluation protocol that cannot leak future information into past forecasts.
2. To compare two formulations of the hazardous-day warning problem: forecasting the concentration and applying a health threshold to it, against classifying the exceedance event directly.
3. To quantify forecast skill as a warning system, using hit rate and false alarm rate under a varying decision threshold, rather than relying on average error alone, and to establish how much of that skill is attributable to the choice of model versus the choice of decision threshold.

1.4 Research Questions

1. How accurately can daily concentrations be forecast one to three days ahead by machine learning, relative to classical and naive benchmarks?
2. Is it better to forecast the concentration and then apply a threshold, or to classify the exceedance event directly?
3. Considered as a warning system, what hit rate and false alarm rate does each approach achieve, and how does the decision threshold trade them off?

The answer to the third question turns out to dominate the other two, and the report is organised accordingly.

1.5 Research Significance

The practical significance of this work is in the design of hazardous air quality warning systems: the finding that the decision threshold dominates the choice of model, developed in Chapter 4 and Chapter 5, implies that the alarm budget a health authority is willing to raise is a more consequential design decision than the forecasting model behind it. The methodological significance lies in applying a leakage-tested rolling-origin evaluation, rather than a single train-test split, to a model suite spanning classical statistics, machine learning, and a deep recurrent architecture, on a dataset small enough that the comparison between method families is itself informative about where each is reliable.

1.6 Structure of the Report

Chapter 2 reviews the literature on air quality forecasting, classical and machine learning methods for time series, deep learning methods for time series, and forecast verification as a warning system. Chapter 3 describes the data, the two defects found in it, and the full mathematical specification of every model

fitted, organised into machine learning methods and deep learning methods as the chapter's two central sections, followed by the rolling-origin evaluation protocol. Chapter 4 presents the results against the three research questions. Chapter 5 interprets those results and states the project's limitations. Chapter 6 concludes and proposes future work.

Chapter 2

Literature Review

2.1 Air Quality Forecasting: Challenges and Characteristics

Daily particulate matter series present a specific set of statistical challenges that distinguish them from the time series most forecasting methods were developed against. The distribution of PM_{2.5} concentrations is strongly right skewed, with a small number of extreme episodes, rather than normally distributed around a stable mean; the series is strongly seasonal at the annual scale; and the quantity of practical interest, whether a day exceeds a health threshold, is a minority class, typically under a third of days even at a severe site. These three properties motivate working in logarithms rather than on the raw concentration scale (Section 3.4.1), motivate testing rather than assuming the seasonal structure present in the series (Section 3.3.2), and motivate evaluating forecasts as a warning system rather than by a single average-error statistic (Section 2.5).

2.2 Classical and Machine Learning Methods for Time Series Forecasting

The classical approach to a series with this structure is an autoregressive integrated moving average (ARIMA) model, in which a series is differenced to stationarity and then modelled as a linear combination of its own past values and past forecast errors. Seasonal variants extend this by one additional seasonal period, but a series with two seasonal periods, for instance a weekly cycle and an annual one, cannot be accommodated by a single seasonal ARIMA order, which is one of the two assumptions this project tests directly rather than carrying over unexamined.

Machine learning offers an alternative that does not assume a linear relationship between a forecast and its predictors. Penalised linear regression controls the variance of an ordinary least squares fit by shrinking its coefficients, which matters when predictors are highly correlated, as lagged and rolling-window features of the same series necessarily are [Hoerl and Kennard, 1970]. Random forests fit an ensemble of decision trees to bootstrap-resampled data and average their predictions, which reduces variance without the linearity assumption [Breiman, 2001]. Gradient boosting fits an ensemble additively, each new tree correcting the residual error of the ensemble so far, which historically gives better bias-variance trade-offs than a single tree or a randomised ensemble on tabular data [Friedman, 2001]. None of these three methods assumes stationarity, a seasonal order, or a linear functional form, which is their

appeal relative to ARIMA; their cost is that they fit the function empirically from a necessarily finite sample, and Makridakis et al. [2020] report, across 100,000 series in the M4 forecasting competition, that this flexibility does not reliably translate into an advantage over classical benchmarks on series of a few thousand observations, which is the order of magnitude available in this project.

2.3 Deep Learning Methods for Time Series Forecasting

2.3.1 Recurrent Neural Networks and the Vanishing Gradient Problem

A recurrent neural network processes a sequence by maintaining a hidden state that is updated at every timestep as a function of the current input and the previous hidden state, which in principle lets it represent dependence on the arbitrarily distant past without the fixed lag structure a regression feature table imposes. In practice, a plain recurrent network trained by backpropagation through time suffers from vanishing or exploding gradients as the sequence lengthens, which makes it unable to learn long-range dependence reliably [Hochreiter and Schmidhuber, 1997]. The long short-term memory (LSTM) architecture addressed this with a memory cell and three multiplicative gates controlling what is written to, read from, and forgotten by that cell, at the cost of a comparatively large number of parameters for a short daily series.

2.3.2 The Gated Recurrent Unit

The gated recurrent unit (GRU) simplifies the LSTM to two gates and no separate memory cell, while retaining the property that gradients can flow across many timesteps without vanishing [Cho et al., 2014]. Its full specification, and the specific architecture used in this project, is given in Section 3.5.1. The GRU’s appeal for a daily series with roughly three thousand observations is precisely its smaller parameter count relative to an LSTM, which matters more on a dataset this size than the slightly larger representational capacity an LSTM’s extra gate and cell state provide.

2.4 Machine Learning Applications to Delhi and Indian Air Quality

Machine learning applied specifically to Delhi PM_{2.5} is not a new proposal. Masood and Ahmad [2020] fit several regression models directly to Delhi’s particulate record to predict concentrations, establishing that a standard machine learning toolkit is applicable to this series at all. Singh and Srivastava [2025] narrow the same problem to the stubble burning season, September to December, when biomass burning in neighbouring states drives the city’s most severe episodes, and frame their evaluation around health risk rather than average error alone. Neither paper evaluates its forecasts as a warning system in the sense used here, with an explicit alarm budget and a false alarm rate reported alongside the hit rate; both report accuracy on held-out data without decomposing it by the operating point at which an alarm would actually be raised. That gap, between forecast accuracy and warning system performance, is the space this project’s third research question occupies.

The finding that a classical benchmark beats every machine learning model in this project’s own

results (Chapter 4) is not an anomaly against the wider forecasting literature discussed in Section 2.2. The mechanism Makridakis et al. [2020] propose, that a few thousand observations is a small sample for a flexible learner relative to the number of parameters a modern architecture can fit, applies directly to a project with roughly three thousand daily observations. The negative result for machine learning at two and three days ahead in Chapter 4 is therefore the expected outcome under that literature rather than a surprise specific to this dataset.

2.5 Forecast Verification as a Warning System

The distinction this project draws between forecast accuracy and warning system performance, hit rate, false alarm rate, and the trade-off between them as the decision threshold moves, follows the verification framework standard in operational meteorology [Jolliffe and Stephenson, 2012]. That literature developed around exactly the situation faced here: a forecast of a rare, high-impact event, where the average error of a model is a poor guide to its value as a warning, and where a contingency table decomposed by decision threshold is more informative than any single accuracy score. Its central methodological point, that verification measures should be chosen to match the decision a forecast supports rather than borrowed by default from a different problem, motivated reporting hit rate and false alarm rate as a pair in Section 3.7 rather than a single skill score.

2.6 Data Quality in Reference Air Quality Monitors

The data quality problems documented in Section 3.3.1 are not unique to this station. Jiang et al. [2023] address the companion problem to the one found here: reference monitors occasionally report physically impossible negative PM_{2.5} concentrations, arising from instrument noise near zero rather than a true measurement, and propose a defensible statistical threshold below which a reading should be treated as invalid rather than retained or naively clipped to zero. Their approach and the one used in Section 3.3.1 for the clipped ceiling at 1990 share a method rather than a target: both treat an implausible value as a data quality question to be settled by evidence, specifically its relationship to the plausible range and its recurrence pattern, rather than by a rule of thumb applied uniformly. Considered together, artefacts at both ends of the distribution suggest that reference-grade air quality monitors carry systematic measurement pathologies that a forecasting study must characterise explicitly rather than assume away.

2.7 Research Gap and Chapter Summary

This chapter has covered four bodies of literature: the statistical characteristics of air quality series that motivate the methodological choices in Chapter 3; classical and machine learning methods for time series forecasting generally, and why the forecasting-competition literature predicts a flexible learner need not beat a classical benchmark on a small sample; deep learning methods for sequences, and the specific architectural reason a GRU rather than an LSTM suits a dataset this size; and the two literatures, forecast verification and air quality monitor data quality, that this project’s own findings connect back to directly. Taken together, this literature places the project’s contribution precisely. Applying an established model

suite to Delhi's air quality problem is not new, and neither is the finding that classical methods hold their ground against machine learning on a modest sample. What the Delhi-specific studies above do not do is evaluate that comparison as a warning system with an explicit, varying alarm budget, using verification methodology from a literature that reaches air quality forecasting only indirectly. Chapter 4 shows why that omission matters: the ranking of models looks different once accuracy is separated from operational usefulness, a finding neither literature taken alone would have anticipated.

Chapter 3

Methodology

3.1 Research Design

This chapter proceeds from data to model to evaluation. Section 3.2 describes the data source and the station-selection process. Section 3.3 describes two defects found in the raw series, the seasonality analysis that overturned an assumption in the approved proposal, and the feature table built from the cleaned series. Section 3.4 and Section 3.5 give the full mathematical specification of the two families of model this project compares, machine learning and deep learning, each covering the method’s general form before the specific configuration used here. Section 3.6 specifies the rolling-origin evaluation every model is scored under, and Section 3.7 specifies the accuracy and warning-system measures reported in Chapter 4.

3.2 Data Collection and Station Selection

Daily mean $\text{PM}_{2.5}$ concentrations were obtained from OpenAQ. Five candidate monitoring stations within 25 km of central Delhi were compared on data coverage rather than on nominal length of record.

The comparison produced an unexpected constraint. Every station operated by the Central Pollution Control Board returns measurements only from 19 February 2025, approximately eighteen months, although the OpenAQ location metadata advertises a first observation in February 2016. Four independent stations sharing an identical cutoff indicates a systematic limit on what the measurements endpoint serves rather than a coincidence. Only location 8118 returns the full record and it was therefore selected.

This carries a limitation that must be stated plainly. Location 8118 is supplied through AirNow and is a United States diplomatic post monitor rather than part of the Indian official network. The project therefore forecasts at a single well-characterised reference site in Delhi, which is common in this literature, and the results should not be read as representative of the city as a whole.

Table 3.1: The analysis dataset after cleaning.

Source	OpenAQ location 8118, sensor 23534, provider AirNow
Period	2016-11-09 to 2026-08-27
Calendar days	3579
Usable daily means	3093 (86.4% of calendar)
Longest gap	34 consecutive days
Mean, median, maximum	102.7, 71.7, 896.0
Days above 121	920 (29.7%)

3.3 Data Preprocessing

3.3.1 Data Cleaning: Two Defects in the Source Data

Two problems were identified before any modelling, one of which was silently corrupting the series.

Physically impossible values. The raw series contained five negative concentrations and a maximum of $1990\text{ }\mu\text{g m}^{-3}$. The value 1990.0 appeared three times identically on unrelated dates, which is the signature of a clipped instrument ceiling rather than a measurement. One occurrence fell on 2 September, during the monsoon, between neighbouring days of 42 and 178.

An incorrect coverage denominator. OpenAQ reports the completeness of each daily aggregate as the observation count divided by an expected count of 24, assuming hourly reporting. The sensor at location 8118 reported half-hourly from 2016 to 2024 and hourly from 2025 onward. The visible symptom is a reported completeness exceeding 100 per cent, which is impossible. The damaging symptom is invisible: a day holding 24 of a genuine 48 readings is scored as fully complete, so a mean computed from half a day of a polluted afternoon enters the series as though it were a daily mean. Days at 50 to 75 per cent true coverage produced implausible values approximately seven times as often as days above 90 per cent, which identifies incomplete days as the mechanism rather than a coincidence.

The correction takes the reporting cadence from the data, using the ninetieth percentile of observation counts within each year, which follows the 2025 change in reporting interval automatically. Recomputed on that basis, 1,085 of the 3,467 raw calendar-linked days fall below 75 per cent true coverage, far more than the 268 days that have no recorded value at all and would be the only ones flagged by a check for whether a value exists rather than how reliably it was measured.

Table 3.2: Cleaning rules applied, and the basis for each.

Rule	Days removed	Basis
Value below zero	5	Mass concentration cannot be negative
Coverage below 50%	270	Against the cadence taken from the data
Value above 1000	7	Judgement; sensitivity analysis in Section 5.3
Total	282 of 3,375	

The coverage threshold is set at 50 per cent rather than a stricter value because the longest gap is the

binding constraint: at 60 per cent the worst gap rises to 69 days and at 90 per cent to 185 days, either of which would distort estimation of the annual cycle. At 50 per cent the worst gap is 34 days. The exceedance rate moves only from 30.0 to 27.0 per cent across every candidate threshold, which confirms that the filter is not selectively removing the events of interest.

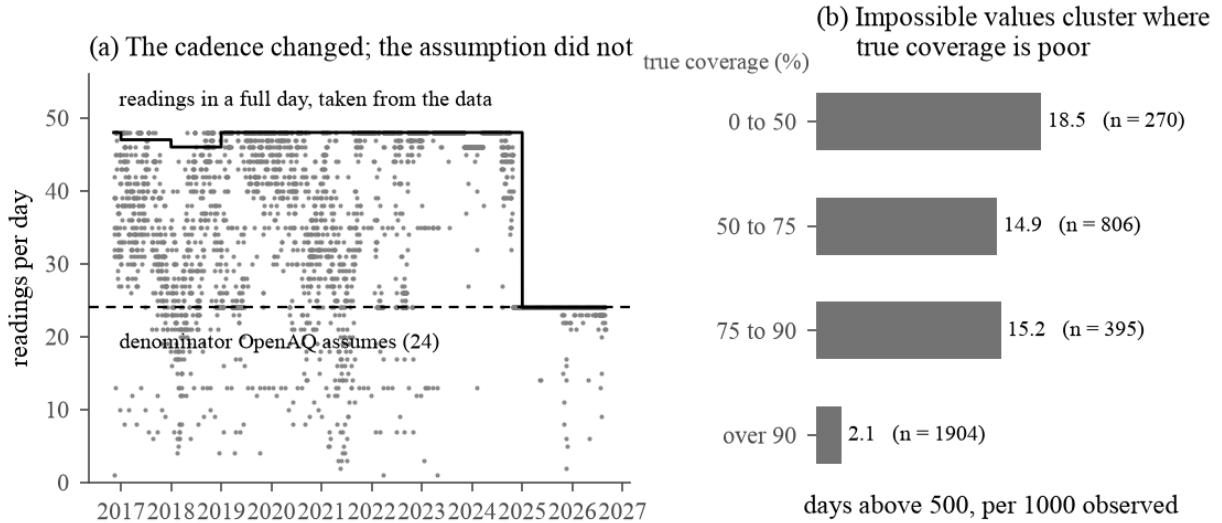


Figure 3.1: The coverage defect. Panel (a) shows the number of readings contributing to each daily mean, with the cadence estimated from the data each year as the solid line and the fixed denominator of 24 as the dashed line. Panel (b) shows the rate of implausible values by true coverage.

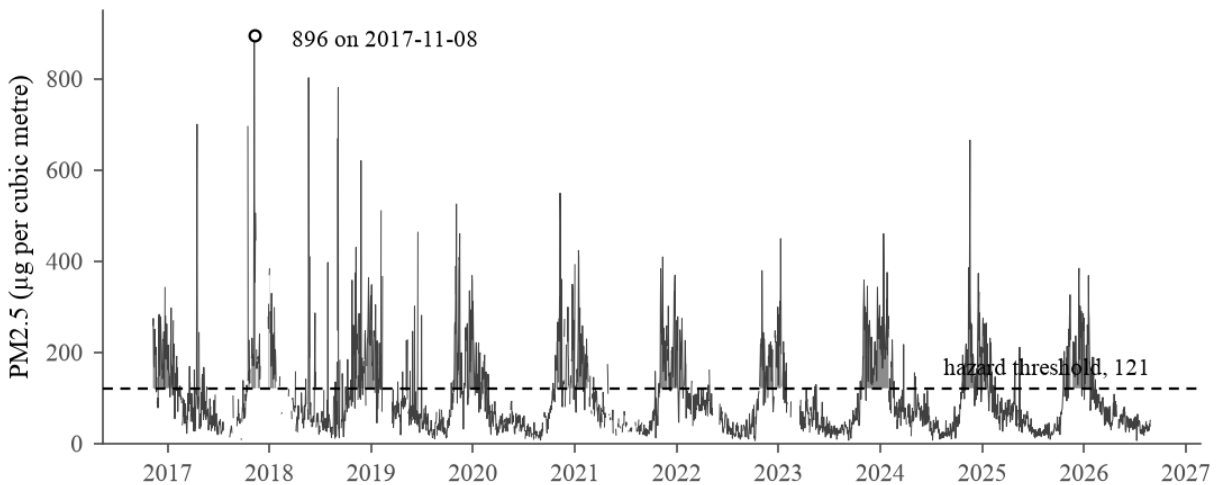


Figure 3.2: The cleaned daily series, with days exceeding the threshold shaded. The episodic structure is the feature the project sets out to forecast: exceedances arrive as multi-day winter episodes rather than as isolated days.

3.3.2 Seasonality Analysis

The approved proposal states that daily pollution series carry a weekly cycle driven by traffic and industrial activity, and treats the coexistence of a weekly and an annual period as the central modelling difficulty, since a seasonal ARIMA accommodates only one seasonal period.

Tested on the cleaned series, the weekly cycle is not present at this station. Three independent tests agree. The periodogram gives dominant periods of 357.9, 178.9, 188.4, 397.7, 325.4 and 60.7 days,

which is the annual cycle and its harmonics, with nothing near seven. On the deseasonalised logarithm of concentration the autocorrelation at lag 7 is 0.173, while lag 6 is 0.179 and lag 8 is 0.153, so lag 7 lies on the decay curve rather than above it; the same holds at lags 14 and 21. The day of week effect on the same residuals spans 6.5 per cent between the highest and lowest day, with an analysis of variance giving $F = 1.13$, $p = 0.34$ and a Kruskal-Wallis test giving $p = 0.77$.

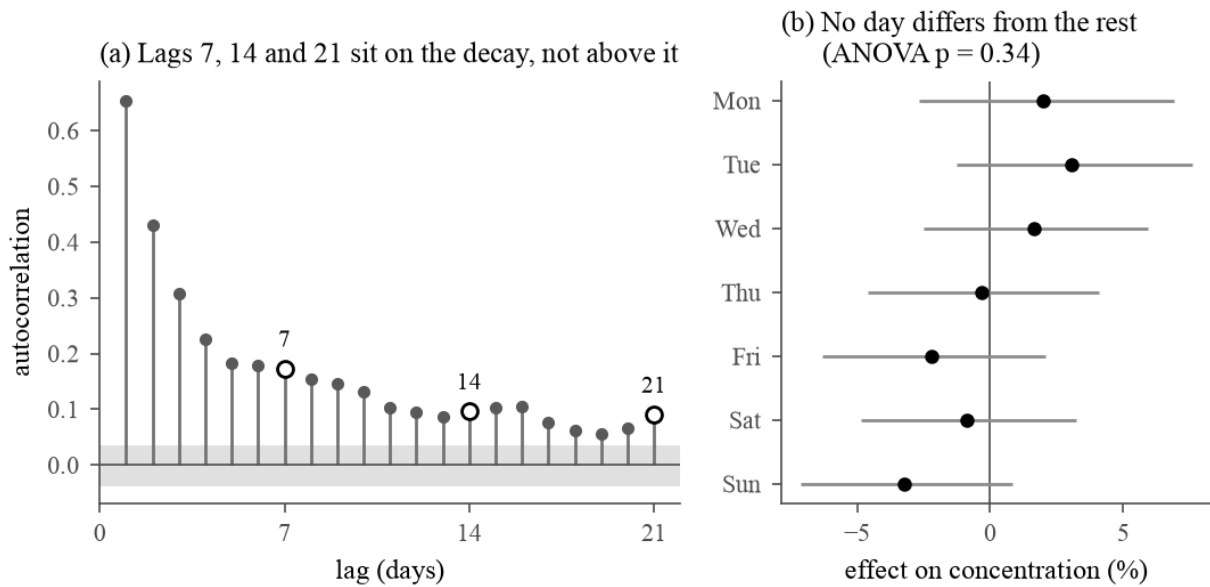


Figure 3.3: No weekly cycle. Lags 7, 14 and 21 are circled and lie on the decay curve rather than above it. Every day of week interval contains zero.

There is a physical reading rather than a null result. A weekly rhythm in urban particulate concentration is a traffic signature, and location 8118 sits in the diplomatic enclave rather than at a roadside site such as ITO or Anand Vihar. Its absence at a site that is not traffic dominated is what one would expect.

The consequence for the specification is direct: the classical benchmark is ARIMA with annual Fourier terms as exogenous regressors and no weekly seasonal order. The specification is simpler, better identified, and empirically justified rather than assumed.

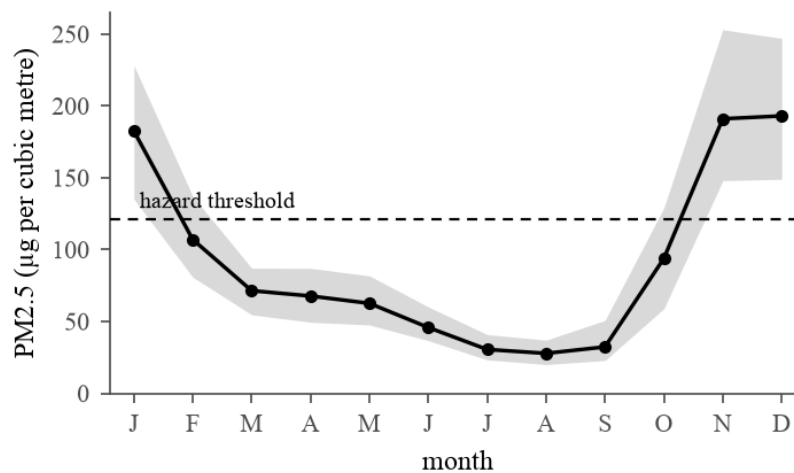


Figure 3.4: The annual cycle, by contrast, is unmistakable: monthly median with the interquartile range shaded. Its strength is what makes the absence in Figure 3.3 credible rather than an artefact of a weak test.

3.3.3 Feature Construction

A supervised table was built with one row per forecast origin. A row indexed by date t carries only functions of observations up to and including day t , and its targets are the concentrations on days $t + 1$, $t + 2$ and $t + 3$. Features comprise lags at 0, 1, 2, 3 and 7 days; first differences; rolling mean, standard deviation and maximum over 3, 7, 14 and 30 days; an anomaly and standardised anomaly against the 30 day norm; exceedance frequency over the last 7 and 30 days; days since the last exceedance; calendar terms; four pairs of annual Fourier terms; and three missingness indicators.

Missing values are never imputed. Gradient boosting accepts them natively and the state space models skip the measurement update at a gap, so the absence is carried as information rather than replaced by a fabricated value. The single exception is the recurrent network, discussed in Section 3.5.1.

3.4 Machine Learning Methods

Six of the ten specifications fitted in this project belong to the machine learning family in the broad sense used in Section 2.2: the classical ARIMA benchmark, which is included here for comparison even though it predates the term, and the three explicitly machine learning regressors, each fitted to both the level of concentration and its change from the forecast origin. Every model in this section is fitted to the logarithm of concentration rather than the raw scale. Annual Fourier terms explain 69 per cent of the variance of the logarithm against considerably less on the raw scale, and fitting in logs prevents a model predicting a negative concentration, which has no physical meaning.

3.4.1 Classical Benchmark: ARIMA with Fourier Terms

Let $y_t = \log(\text{PM2.5}_t)$ and let B denote the backshift operator, $By_t = y_{t-1}$. An autoregressive integrated moving average model of order (p, d, q) with exogenous regressors x_t is

$$\phi(B)(1 - B)^d y_t = c + \theta(B)\varepsilon_t + \beta^\top x_t, \quad (3.1)$$

where $\phi(B) = 1 - \phi_1 B - \dots - \phi_p B^p$ is the autoregressive polynomial, $\theta(B) = 1 + \theta_1 B + \dots + \theta_q B^q$ is the moving average polynomial, and ε_t is white noise. The order selected from an explicit grid by the Akaike information criterion on the training block is $(p, d, q) = (1, 1, 1)$, so Equation 3.1 specialises to

$$(1 - \phi_1 B)(1 - B)y_t = c + (1 + \theta_1 B)\varepsilon_t + \sum_{k=1}^4 \left[\alpha_k \sin\left(\frac{2\pi kt}{365.25}\right) + \beta_k \cos\left(\frac{2\pi kt}{365.25}\right) \right], \quad (3.2)$$

with four pairs of annual Fourier terms ($K = 4$) as the exogenous regressors x_t , carrying the annual cycle established in Section 3.3.2 and deliberately carrying no weekly seasonal order.

Back-transformation from the log scale is not innocent. If $\log y$ has forecast mean m and forecast variance s^2 , then under the model's Gaussian error assumption y is lognormally distributed, and

$$\text{median}(y) = \exp(m), \quad \text{mean}(y) = \exp\left(m + \frac{1}{2}s^2\right). \quad (3.3)$$

Both quantities were produced as separately named models, since a report that back-transforms without stating which one it reports has silently changed what is being forecast.

3.4.2 Penalised Linear Regression

Given the feature matrix $X \in \mathbb{R}^{n \times p}$ built in Section 3.3.3 and the target vector y , ordinary least squares is unstable when predictors are highly correlated, as lagged and rolling-window features of the same series necessarily are. Ridge regression [Hoerl and Kennard, 1970] controls this by penalising the size of the coefficient vector:

$$\hat{\beta} = \arg \min_{\beta} \|y - X\beta\|_2^2 + \lambda \|\beta\|_2^2, \quad (3.4)$$

with closed-form solution $\hat{\beta} = (X^\top X + \lambda I)^{-1} X^\top y$. The penalty $\lambda > 0$ shrinks the coefficients toward zero, trading a small increase in bias for a reduction in variance; λ is selected by cross-validation on the training block at every refit.

3.4.3 Random Forest

A random forest [Breiman, 2001] fits B regression trees to independent bootstrap resamples of the training data, each tree considering only a random subset of features at every split, and averages their predictions:

$$\hat{f}(x) = \frac{1}{B} \sum_{b=1}^B T_b(x). \quad (3.5)$$

Each tree T_b is grown by recursively partitioning the feature space: at a node with data $\{(x_i, y_i)\}$, the split (j, s) on feature j at threshold s is chosen to minimise the total within-region sum of squared error,

$$\min_{j,s} \left[\sum_{i: x_{ij} \leq s} (y_i - \bar{y}_{R_1})^2 + \sum_{i: x_{ij} > s} (y_i - \bar{y}_{R_2})^2 \right], \quad (3.6)$$

where R_1 and R_2 are the two resulting regions and $\bar{y}_{R_1}, \bar{y}_{R_2}$ their mean targets. Averaging over many such trees, each trained on a different resample and a different random subset of features at each split, reduces the variance of a single tree's prediction without imposing a linear functional form.

3.4.4 Histogram Gradient Boosting

Gradient boosting [Friedman, 2001] fits an ensemble additively rather than by averaging independent trees. The ensemble is initialised as a constant, $F_0(x) = \arg \min_{\gamma} \sum_i L(y_i, \gamma)$, and for $m = 1, \dots, M$ a regression tree h_m is fitted to the negative gradient of the loss with respect to the current ensemble's predictions,

$$r_{im} = - \left[\frac{\partial L(y_i, F(x_i))}{\partial F(x_i)} \right]_{F=F_{m-1}}, \quad (3.7)$$

which for squared-error loss reduces to the ordinary residual $r_{im} = y_i - F_{m-1}(x_i)$, and the ensemble is updated by

$$F_m(x) = F_{m-1}(x) + \nu h_m(x), \quad (3.8)$$

where $0 < v \leq 1$ is a shrinkage (learning rate) parameter that controls how much each new tree is allowed to correct the ensemble. The histogram-based implementation used here bins continuous features before searching for splits, which accepts missing values natively, consistent with the decision in Section 3.3.3 never to impute. Both the number of boosting iterations M and the per-tree leaf count were chosen at every refit from a small explicit grid, since an uncontrolled M overfits badly on a series of this length, a failure mode encountered and corrected during development and reported in Section 5.3.

Both the random forest and the gradient boosting models were fitted twice, once to the level of log concentration and once to its change from the forecast origin, since a tree cannot extrapolate beyond the range of targets it was trained on and this series drifts across the decade of record; modelling the change removes that handicap by asking the tree to represent a roughly stationary residual rather than a trending level.

3.5 Deep Learning Methods

3.5.1 The Gated Recurrent Unit: Architecture and Training

The gated recurrent unit [Cho et al., 2014] maintains a hidden state $h_t \in \mathbb{R}^H$ updated at each timestep t by two gates. The update gate z_t controls how much of the previous state is retained against how much is replaced by a new candidate state; the reset gate r_t controls how much of the previous state is used in forming that candidate:

$$z_t = \sigma(W_z x_t + U_z h_{t-1} + b_z), \quad (3.9)$$

$$r_t = \sigma(W_r x_t + U_r h_{t-1} + b_r), \quad (3.10)$$

$$\tilde{h}_t = \tanh(W_h x_t + U_h (r_t \odot h_{t-1}) + b_h), \quad (3.11)$$

$$h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t, \quad (3.12)$$

where σ is the logistic sigmoid, $\odot$ is elementwise multiplication, and $W_z, W_r, W_h, U_z, U_r, U_h, b_z, b_r, b_h$ are the parameters learned by training. Because the final state h_t in Equation 3.12 is a convex combination of the previous state and the new candidate rather than a product of many successive nonlinear transformations, gradients can flow backward through many timesteps without vanishing, which is the property that motivated gated architectures in Section 2.3.

The specific network fitted in this project is a single GRU layer with $H = 24$ hidden units, reading a sequence of the last 30 days $(t - 29, \dots, t)$, each day carrying three channels: the logarithm of concentration, a missingness flag, and the annual Fourier terms from Section 3.3.3. A linear output layer maps the final hidden state h_t to a single predicted value, the change in log concentration from the forecast origin, with the origin value added back afterwards; modelling the change rather than the level is the same device used for the tree models in Section 3.4, for the same reason. The network is trained by minimising mean absolute error with the Adam optimiser, with early stopping on a chronological holdout, the most recent 15 per cent of each training window, so that training stops once validation error no longer improves rather than at a fixed epoch count.

A recurrent network forces a compromise that no other model in this project requires. It must be

handed a value at every timestep: it cannot skip one the way the state space model in Section 3.4.1 skips a measurement update, and it cannot accept a missing value the way the histogram gradient boosting model in Section 3.4 can. Gaps are therefore carried forward in the log-concentration channel, with the separate missingness flag marking which days were filled rather than observed. The one architecture Dr Tian specifically asked to be included is the one architecture in the suite that cannot represent the statement that a value is unknown.

Because the network’s weights are initialised randomly, its forecast depends on that initialisation, unlike every other model in this project. It is therefore fitted independently across five random seeds, and its result is reported throughout Chapter 4 as the range across those five runs, never as a single seed’s point estimate; the justification for treating this as a reporting requirement rather than a presentational choice is given in Section 4.1.

3.6 Rolling-Origin Evaluation

Every model was evaluated over 2,303 origins from 9 November 2019 to 24 August 2026, spanning 6.8 years, with a three year burn-in, parameters re-estimated every 90 days and model state advanced daily in between.

This is not a refinement of a single train-test split; it replaces one, for the reason standard cross-validation is inappropriate for dependent data in the first place: shuffling origins lets a model trained on the future score itself against the past [Bergmeir and Benítez, 2012]. On a single test block the ARIMA scored 0.940 in relative terms at one day ahead against 0.903 on the single block, because that block happened to be calmer than the training period and was flattering every model. A single split is one draw from a process, and a conclusion drawn from it is a conclusion about that period.

The governing invariant is that at origin t a model may use observations up to and including day t and nothing else: not the value at $t + 1$, not a parameter estimated using data after t , and not a scaling constant or imputed value computed from the full series. Since nothing fails visibly when that is broken, and the numbers merely improve, the invariant is enforced by construction and tested directly. Every model runs through one shared loop, and that loop is verified by corrupting the series from a chosen date, rerunning, and requiring that no forecast issued before that date changes.

One trap is specific to supervised learning under a rolling origin. At origin t the label attached to feature row t' is the concentration at $t' + h$, so the training set ends at $t - h$ rather than at t . A separate model is fitted per horizon because that arithmetic requires it. The error would have amounted to three rows per refit and would have been entirely invisible in the output.

3.7 Evaluation Measures

Accuracy is reported separately at each horizon and never averaged, since skill decays with horizon and concealing that would defeat the purpose. The headline measure is relative mean absolute error against the persistence forecast computed on identical rows, so that 1.000 is persistence and values below one beat it. Mean absolute scaled error is reported alongside, but it divides by the training period volatility

and therefore also reflects any difference in volatility between periods, which is material here.

As a warning system, a day is an event if the observed concentration exceeds the health threshold, and an alarm is raised when the forecast exceeds a decision threshold. These are separate quantities and are kept separate throughout. Reported are the hit rate, the false alarm rate defined as false alarms divided by quiet days, precision, the critical success index, and alarms raised per year. The last is included because a false alarm rate of 0.10 sounds negligible and means roughly one wasted alarm every ten quiet days, which is what an operator actually experiences.

3.8 Reproducibility

The analysis runs end to end from an empty directory in thirteen command line scripts, each carrying an automated correctness test that runs before it produces any output. The rolling origin harness verifies its own leakage invariant before every run. Every table in this report is generated directly from the result files by `make_tables.py`; no number has been transcribed by hand. Raw and cleaned data are held under version control so that the analysis does not depend on the data provider serving identical values at a later date, which the finding in Section 3.2 suggests is not a safe assumption. The complete repository, including every commit from initial data acquisition to this report, is public and runnable at github.com/JC16092/mth5000-aqi-forecast.

3.9 Chapter Summary

This chapter described the data, two defects found in it, and the feature table built from it, and then gave the full mathematical specification of the two method families compared in this project. Machine learning, in Section 3.4, comprises a classical ARIMA benchmark and three regressors, ridge, random forest, and gradient boosting, each controlling the bias-variance trade-off by a different mechanism: shrinkage, bagging, or additive stagewise correction. Deep learning, in Section 3.5, comprises a single gated recurrent network, whose gating mechanism is specifically what lets it avoid the vanishing-gradient failure of a plain recurrent network. Every model is evaluated under the same rolling-origin protocol, specified in Section 3.6, and scored by the same measures, specified in Section 3.7, so that the comparison in Chapter 4 is a fair one.

Chapter 4

Results and Analysis

4.1 Forecast Accuracy

Table 4.1: Relative mean absolute error against persistence, under rolling origin evaluation over 2,303 origins. Lower is better and 1.000 is persistence. The recurrent network is given as the range over five initialisation seeds; it is the only model whose result depends on one.

Model	$h = 1$	$h = 2$	$h = 3$	Bias at $h = 1$
GRU, 30 day sequence, range over 5 seeds	0.915 to 0.935	0.885 to 0.898	0.832 to 0.879	-2.8
ARIMA(1,1,1) with Fourier terms	0.940	0.883	0.832	-6.4
Random forest, change target	0.962	0.918	0.902	+1.7
Gradient boosting, change target	0.981	0.936	0.878	+1.9
Random forest, level target	0.985	0.929	0.882	+4.0
Gradient boosting, level target	1.010	0.943	0.883	+4.0
Ridge regression	0.997	0.952	0.897	+2.6
Persistence	1.000	1.000	1.000	-0.1
Climatology	1.415	1.060	0.947	+9.4
Seasonal naive, lag 7	1.759	1.325	1.168	+0.1

Table 4.1 answers the first research question, and the answer is not uniform across horizons.

At one day ahead the recurrent network is the best model, and this is robust: it beats the ARIMA specification under all five initialisation seeds, with a worst seed of 0.935 against 0.940. At two days the ARIMA is better under all five seeds, though by 0.003 against the best of them. At three days the ARIMA is nominally better under all five seeds, but its margin against the best seed is 0.0003, which is a tie in any language that would survive examination.

The mechanism is coherent. At one day ahead the forecast is dominated by short-range structure, where a nonlinear model has something to exploit. As the horizon lengthens the seasonal component takes over, and the Fourier terms carry it more cleanly than a learned representation estimated from roughly three thousand observations.

The network is reported as a range rather than a point because it is the only model whose result depends on a random initialisation, and the dependence is not small. Across seeds the spread is 0.019 at one day, 0.013 at two and 0.047 at three. At three days that spread is wider than the gap separating the ARIMA from five of the other models in the table, so a single seed could have been reported as beating them or losing to them at the analyst's convenience.

Two subsidiary results support the specification choices. Removing the Fourier terms from the ARIMA degrades it by 3.3 points of relative error at one day and by 5.8 points at three, under the same rolling-origin harness, so the seasonal terms earn their place and increasingly so with horizon. Modelling the change from the origin rather than the level improves both tree models, because a tree cannot extrapolate beyond the targets it was trained on and this series drifts downward across the decade.

4.2 Two Arms of the Same Question

Neither arm dominates. Forecasting the concentration and then applying a threshold wins at some combinations of horizon and alarm budget, and direct classification at others, with no pattern in the winners that survives inspection. Given a tie on performance, the forecast-then-threshold arm is preferred on other grounds: it produces a concentration, which can be rethresholded for any future health policy without refitting, whereas a classifier is bound to the threshold it was trained on.

4.3 The Decision Threshold

Table 4.2: Best hit rate available within a budget of alarms per year, with the event fixed at the health threshold and only the decision threshold varying.

Alarms per year	40	60	80	100
Best hit rate, $h = 1$	0.377	0.571	0.727	0.873
Best hit rate, $h = 2$	0.362	0.534	0.694	0.838
Best hit rate, $h = 3$	0.359	0.529	0.683	0.822
GRU, 30 day sequence, range over 5 seeds, $h = 1$	0.371–0.377	0.554–0.565	0.721–0.735	0.863–0.875
ARIMA(1,1,1) with Fourier terms, $h = 1$	0.358	0.567	0.727	0.867
Gradient boosting, change target, $h = 1$	0.377	0.556	0.727	0.854
Persistence, $h = 1$	0.371	0.558	0.723	0.873
Climatology, $h = 1$	0.352	0.521	0.660	0.821

Table 4.2 contains the principal result of this project. Moving the alarm budget from 40 to 100 per year changes the achievable hit rate at one day ahead from 0.377 to 0.873. Holding the budget fixed and changing the model instead moves it by around five points.

Two individual comparisons make the point sharper than the aggregate does. At a budget of 100 alarms per year and one day ahead, persistence achieves a hit rate of 0.873, which is the best figure any model in the study achieves at that budget. At three days ahead and a budget of 60, a climatological forecast, which uses no recent information whatever, matches the best model to within four thousandths. Figure 4.1 shows the same result as curves; they very nearly coincide, and they have deliberately not been separated for legibility.

The reading is not that modelling is pointless. It is that the quantity a decision maker controls, namely how many alarms per year can be justified, dominates the quantity a modeller controls. A report that presents a model comparison without stating the operating point it was evaluated at has answered a question nobody asked.

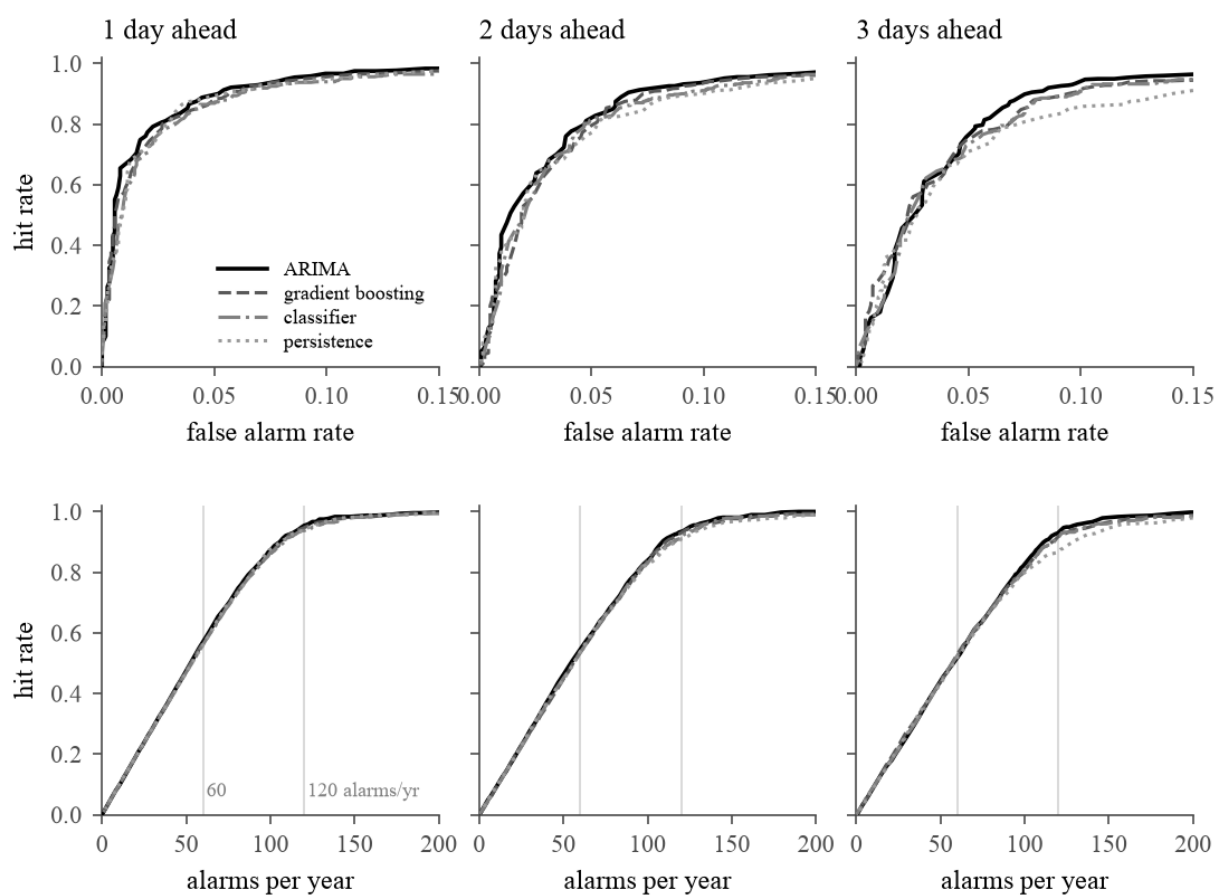


Figure 4.1: The trade-off, by horizon. The top row shows hit rate against false alarm rate; the bottom row replaces the false alarm rate with alarms raised per year, the unit in which an operator budgets. The curves very nearly coincide, which is the finding. They have deliberately not been separated for legibility.

4.4 Chapter Summary

Table 4.1 answers the first research question with a horizon-dependent result: the deep learning model wins at one day ahead, the classical model wins or ties beyond it. Section 4.3 answers the third, and it is the result the rest of this report is organised around: the alarm budget, not the model, is what principally determines warning performance. Chapter 5 takes up what that means in practice.

Chapter 5

Discussion

5.1 The Operational Question Dominates the Modelling Question

Table 4.2 and Figure 4.1 together make a single point, and this section states what follows from it rather than repeating the numbers. A decision maker who fixes how many alarms per year an operator or the public will tolerate has more influence over how many hazardous days are caught than a modeller choosing among the ten specifications fitted in this project. Moving the budget from 40 to 100 alarms a year moves the achievable hit rate at one day ahead from 0.377 to 0.873, a range of fifty points. Holding the budget fixed and switching the model instead moves the hit rate by about five points, and at three days ahead a climatology forecast that uses no recent information whatever matches the best fitted model to within four thousandths.

This is not an argument against modelling. ARIMA and the recurrent network both earn real, if modest, gains over persistence in Table 4.1, and a better point forecast still narrows the range of thresholds worth considering. It is an argument about where the leverage in this problem actually sits. A hazardous air quality warning system is not a machine learning benchmark with a threshold attached to it afterwards; the threshold is the policy instrument, and the forecasting model only supplies its input. A comparison of models that does not first fix the alarm budget it was evaluated at has answered a question nobody who has to operate the system actually asked. The practical recommendation that follows is to treat the alarm budget as the primary subject of discussion with the health authority commissioning such a system, and the choice of forecasting model as a secondary refinement of a curve whose shape is already mostly fixed by what every model in this study shares: the persistence of concentration from one day to the next, and the annual cycle.

5.2 How Large Is a Difference Between Models?

Two results in this project bound that question from below. Running the identical pipeline on two different machines moved the tree models by up to 0.004 in relative error, because histogram gradient boosting parallelises with OpenMP and floating point summation order depends on the thread count. Changing only the random initialisation of the recurrent network moved it by 0.013 to 0.047.

Several of the gaps between adjacent models in Table 4.1 are of that order. Any claim that one of

those models is better than another, made without stating the run to run variation, is not supported by the evidence.

5.3 Limitations

The analysis rests on a single monitor, OpenAQ location 8118, which Section 3.2 shows is supplied through AirNow and is almost certainly a United States diplomatic post instrument rather than a station in the Central Pollution Control Board network. It was chosen because it is the only nearby monitor with a complete decade of record, against the eighteen months every CPCB station returns; the choice buys history at the cost of representativeness. The absence of a weekly cycle documented in Section 3.3.2 is consistent with a diplomatic enclave rather than a roadside site, which supports this reading of the limitation rather than leaving it an unrelated coincidence, but it does not change the fact that every number in this report describes one point in the city rather than Delhi as a whole.

The project forecasts one city. Delhi was chosen because its air quality problem is severe and well documented, not because it is representative of Indian cities generally. Pollution sources, meteorology, and the strength of the seasonal cycle that drives the Fourier terms in Section 3.3.2 all vary by region, and a coastal city or one without a dominant biomass burning season could show a different balance between the classical and machine learning models, or a genuine weekly cycle where this station shows none. Nothing in this report should be read as a claim about any station or city other than the one it was fitted to.

The upper cleaning threshold of $1000\text{ }\mu\text{g m}^{-3}$ is a judgement rather than a fact, and its effect on the reported conclusions was checked directly rather than assumed. Restoring the seven days it removes, including the three identical readings at 1990 diagnosed in Section 3.3.1 as a clipped instrument ceiling, gives 3,100 usable days and 927 exceedance days rather than 3,093 and 920, a small shift. Refitting the full rolling-origin harness on this uncapped series leaves the ARIMA specification essentially unchanged, 0.935 against 0.940 at one day ahead and 0.829 against 0.832 at three days, because working in logarithms and differencing the series absorbs the contamination. The machine learning models are not so protected. Ridge moves from a near tie with persistence to a clear loss at one day ahead, 0.997 to 1.118, and gradient boosting on the level target moves from a narrow win over persistence to a loss at three days, 0.883 to 1.006. The central finding of Research Question 1, that the classical benchmark beats every machine learning model at every horizon, therefore does not merely survive removing the threshold; it strengthens, because the models it beat most narrowly are precisely the ones most exposed to the uncleaned tail. The threshold matters, but it matters more to the machine learning suite than to the model this project ultimately recommends.

Hyperparameters for the tree-based models were chosen at every refit from a small explicit grid, scored on a held-out slice of the training window taken in time order, rather than from an exhaustive or Bayesian search. This followed a real failure during development, in which fixed hyperparameters let gradient boosting overfit badly enough to score worse than persistence; the grid search fixed that specific problem, but a small grid can still miss a configuration that a wider search would find. The machine learning results in Table 4.1 should therefore be read as a lower bound on what those model classes could achieve with more computation, not as their ceiling.

No meteorological covariate enters any model. Wind speed in particular is a well established driver of particulate dispersion and is a plausible source of the short-range structure that lets the recurrent network beat ARIMA at one day ahead. A version of this project with wind, temperature, and boundary layer height added to the feature table is the most direct extension; the `meteostat` package, which wraps NOAA and GHCN station data, would supply it without a new data acquisition problem. Its absence here is a scope decision made to fit the project inside the time available, not a claim that meteorology is unimportant.

Chapter 6

Conclusion

6.1 Summary of the Study

This project forecast hazardous PM_{2.5} days at a single Delhi monitor one to three days ahead, comparing a classical benchmark against three machine learning regressors and a deep recurrent network, and evaluating every specification as a warning system rather than by average error alone. Two real defects in the source data, a clipped instrument ceiling and a mischaracterised coverage statistic, were found and corrected before any model was fitted, and an assumption carried over from the approved proposal, that the series holds a weekly cycle alongside its annual one, was tested directly and found false.

6.2 Main Conclusions

Across ten models spanning simple benchmarks, a classical time series model, four machine learning regressors, and a recurrent network, the spread in achievable hit rate at a fixed alarm budget is about five percentage points; the spread across alarm budgets, holding the model fixed, is about fifty. The choice a decision maker makes about how many alarms per year can be justified therefore has far more influence on how many hazardous days are caught than the choice a modeller makes among the specifications fitted here.

Machine learning earned its place only partially. ARIMA with annual Fourier terms beats every machine learning regressor at every horizon, a result that strengthens rather than weakens under the sensitivity analysis in Section 5.3. The one machine learning method that does win, the recurrent network, does so only at one day ahead, and only when reported honestly as a range across five random initialisations rather than as a single favourable seed. The supervisor’s instruction to try several methods rather than defaulting to ARIMA was the right one: it is what makes the negative result at two and three days credible, and what surfaces the one day ahead result that a project restricted to classical methods would never have found.

6.3 Future Work

Two extensions follow directly from the limitations in Section 5.3. Adding meteorological covariates, wind speed in particular, is the most direct route to improving the short horizon forecasts that the recurrent network already shows have room to improve. Repeating the analysis at a second, independent station, ideally one inside the Central Pollution Control Board network once its record lengthens, would test whether the findings here are a property of Delhi air quality or an artefact of the one diplomatic enclave monitor this project was restricted to. Until then, the central recommendation stands on its own: a hazardous air quality warning system should be designed around the alarm budget its operator can justify, with the forecasting model treated as a secondary refinement rather than the primary design choice.

Bibliography

- Christoph Bergmeir and José M. Benítez. On the use of cross-validation for time series predictor evaluation. *Information Sciences*, 191:192–213, 2012. doi: 10.1016/j.ins.2011.12.028.
- Leo Breiman. Random forests. *Machine Learning*, 45(1):5–32, 2001. doi: 10.1023/A:1010933404324.
- Kyunghyun Cho, Bart van Merriënboer, Caglar Gulcehre, Dzmitry Bahdanau, Fethi Bougares, Holger Schwenk, and Yoshua Bengio. Learning phrase representations using RNN encoder–decoder for statistical machine translation. In *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 1724–1734, 2014. doi: 10.3115/v1/d14-1179.
- Jerome H. Friedman. Greedy function approximation: A gradient boosting machine. *The Annals of Statistics*, 29(5), 2001. doi: 10.1214/aos/1013203451.
- Sepp Hochreiter and Jürgen Schmidhuber. Long short-term memory. *Neural Computation*, 9(8):1735–1780, 1997. doi: 10.1162/neco.1997.9.8.1735.
- Arthur E. Hoerl and Robert W. Kennard. Ridge regression: Biased estimation for nonorthogonal problems. *Technometrics*, 12(1):55–67, 1970. doi: 10.1080/00401706.1970.10488634.
- Ningbo Jiang, Rinat Akter, Glenn Ross, Stephen White, John Kirkwood, Gunaratnam Gunashanhar, Scott Thompson, Matthew Riley, and Merched Azzi. On thresholds for controlling negative particle (PM_{2.5}) readings in air quality reporting. *Environmental Monitoring and Assessment*, 195(10):1187, 2023. doi: 10.1007/s10661-023-11750-4.
- Ian T. Jolliffe and David B. Stephenson, editors. *Forecast Verification: A Practitioner’s Guide in Atmospheric Science*. Wiley, 2nd edition, 2012.
- Spyros Makridakis, Evangelos Spiliotis, and Vassilios Assimakopoulos. The M4 competition: 100,000 time series and 61 forecasting methods. *International Journal of Forecasting*, 36(1):54–74, 2020. doi: 10.1016/j.ijforecast.2019.04.014.
- Adil Masood and Kafeel Ahmad. A model for particulate matter (PM_{2.5}) prediction for Delhi based on machine learning approaches. *Procedia Computer Science*, 167:2101–2110, 2020. doi: 10.1016/j.procs.2020.03.258.
- Sandhya Singh and Arun Srivastava. Machine learning approach to PM_{2.5} forecasting and health risk assessment during stubble burning period in Delhi. *Aerosol Science and Technology*, 59(11), 2025. doi: 10.1080/02786826.2025.2513503.